

Reinforcement Learning for Blackjack: A Comparative Study of Monte Carlo and Temporal Difference Methods

Name: Kurt Abela, Luke Camilleri

Email: kurt.abela.24@um.edu.mt, luke.camilleri.24@um.edu.mt

Student ID: 0150106L, 0207606L

Study Unit: Reinforcement Learning (ARI2204)

1. Overview and Environment.....	3
1.1. Environment Dynamics.....	3
1.1.1. State Representation.....	3
1.1.2. Game Mechanics and Ace Handling.....	4
1.1.3. Action Space and Rewards.....	4
1.1.4. Exploring Starts Implementation.....	4
2. Algorithm Implementations.....	5
2.1. Monte Carlo On-Policy Control.....	5
2.1.1. Implementation and Logic.....	5
2.1.2. Exploration Configurations.....	5
2.2. SARSA (On-Policy TD).....	6
2.2.1. On-Policy Update Mechanism.....	6
2.2.2. Adaptive Step Size.....	6
2.3. Q-Learning (Off-Policy TD).....	7
2.3.1. Off-Policy Update Logic.....	7
2.3.2. Implementation and Convergence.....	7
2.4. Double Q-Learning.....	8
2.4.1. Mitigation of Maximization Bias.....	8
2.4.2. Dual-Table Update Mechanism.....	8
2.4.3. Policy and Exploitation.....	8
3. Experimental Setup.....	10
3.1. Exploration-Exploitation Configurations.....	10

3.2. Evaluation Metrics.....	11
4. Results and Evaluation.....	12
4.1. Convergence and Stability.....	12
4.1.1. Temporal Difference Agents.....	12
4.1.2. Monte Carlo Agent.....	14
4.2. Comparative Win Rate Analysis.....	15
4.2.1. Impact of Epsilon Schedules.....	15
4.3. Exploration Analysis.....	18
4.3.1. State-Action Coverage.....	18
4.3.2. Visit Distribution (Double Q-Learning Example).....	19
4.3.3. Final Performance Comparison: Dealer Advantage.....	20
5. Strategy Visualization.....	22
5.1. Double Q-Learning.....	22
5.2. Q-Learning.....	23
5.3. SARSA (Decay = $e^{-k/10000}$).....	25
5.4. Monte Carlo.....	27
5.5. Comparative Insights.....	29
6. Conclusion.....	30
7. References.....	31
Distribution of Work.....	32
Plagiarism Form.....	34
Generative AI Appendix.....	35

1. Overview and Environment

The objective of this study is to implement and empirically evaluate the performance of four prominent Reinforcement Learning (RL) algorithms: Monte Carlo (MC) On-Policy control, SARSA, Q-Learning, and Double Q-Learning. These algorithms are applied within a simplified Blackjack environment (*Sutton and Barto, 2018*), providing a framework to analyze the trade-offs between on-policy and off-policy learning, as well as Temporal Difference (TD) versus Monte Carlo methods in a stochastic environment. The primary goal is to determine which approach most effectively converges to an optimal policy while managing the exploration-exploitation trade-off across various ϵ -greedy schedules.

1.1. Environment Dynamics

The Blackjack environment was developed to simulate a simplified single-deck card game (52 cards) between a player and a dealer. It serves as a Markov Decision Process (MDP) where the environment manages the deck, tracks the game state, and enforces the rules governing transitions and terminal rewards.

1.1.1. State Representation

At the commencement of each episode, the player is dealt two cards face up, while the dealer receives a single visible card. The agent observes the environment through a state tuple consisting of three elements:

- A. **Player's Current Hand Total:** An integer value representing the sum of the cards.
- B. **Dealer's Visible Card:** The value of the single card shown by the dealer.
- C. **Usable Ace Flag:** A boolean indicator specifying if the player holds an Ace currently valued at 11 without exceeding 21.

This state representation captures all information visible to the player that is relevant to optimal decision-making.

1.1.2. Game Mechanics and Ace Handling

Ace handling is managed dynamically via a dedicated calculation method. If a hand total exceeds 21 while a "usable" Ace (valued at 11) is present, the Ace is automatically re-valued to 1, reducing the hand total by 10. This logic repeats until the total is 21 or below, or until no further usable Aces remain. This ensures the agent accurately evaluates the "safety net" provided by an Ace. Cards are drawn sequentially from a single pre-shuffled deck, with a fresh deck initialised and shuffled at the start of each episode. This ensures that card distribution is statistically consistent with a real single-deck game.

1.1.3. Action Space and Rewards

The agent can choose from two discrete actions:

- A. **HIT (1)**: The player draws an additional card. If the resulting total exceeds 21, the player "busts," the episode terminates immediately, and the agent receives a reward of **-1**.
- B. **STAND (0)**: The player ceases drawing cards, and the dealer executes its fixed policy (drawing until reaching a total of 17 or higher).

Once the dealer stops, the final totals are compared. A player total higher than the dealer's (without busting) results in a reward of **+1** (Win). A lower total results in **-1** (Loss), and equal totals result in **0** (Draw).

1.1.4. Exploring Starts Implementation

To support the Exploring Starts (ES) variant of the Monte Carlo algorithm, a specialized `reset_to_state` method was implemented. This allows the environment to be initialized into specific, arbitrary states (defined by player total, dealer card, and ace status) rather than relying on natural deals. In Blackjack, certain states (such as player totals between 12 and 20) are statistically less likely to be visited frequently under a standard greedy policy. By forcing these states at the start of training, the agent is guaranteed to explore and learn Q-values for the entire state-action space (Sutton and Barto, 2018), facilitating convergence toward the true optimal policy.

2. Algorithm Implementations

To ensure modularity and consistency across different learning paradigms, all implemented agents inherit from a core `BlackJackAgent` base class. This class provides the fundamental framework for state management and implements an ϵ -**greedy policy** for action selection. Under this policy, the agent selects a random action with probability ϵ to explore the environment and chooses the action with the highest estimated Q-value (the greedy action) with probability $1-\epsilon$. This mechanism is critical for balancing the discovery of new strategies with the exploitation of learned knowledge. To ensure correctness, the policy only consults the Q-table and applies ϵ -greedy selection for player sums in the range $[12, 20]$. For sums below 12, the agent always selects HIT, as there is no risk of busting. For a sum of exactly 21, the agent always selects STAND, as no improvement is possible.

2.1. Monte Carlo On-Policy Control

Monte Carlo methods estimate state-action values $Q(s, a)$ by averaging returns observed across complete episodes. This approach is well-suited for Blackjack, as rewards are sparse and only assigned at the terminal state.

2.1.1. Implementation and Logic

The agent utilizes a `defaultdict` for the Q-table, initializing unseen states to $[0.0, 0.0]$.

Returns: With $\gamma = 1.0$ and terminal rewards, the return G for each visited state-action pair simply equals the final reward ($+1, 0, -1$).

First-Visit Update: Only the first occurrence of a state-action pair in an episode contributes to the update, ensuring the independence of the average return calculation.

2.1.2. Exploration Configurations

Four ϵ -greedy schedules were evaluated:

- A. **Exploring Starts (ES)**: Randomizes the starting state (12–20) and first action.
- B. **Reciprocal Decay**: $\varepsilon = \frac{1}{k}$
- C. **Fast Exponential Decay**: $\varepsilon = e^{\frac{-k}{1000}}$
- D. **Slow Exponential Decay**: $\varepsilon = e^{\frac{-k}{10000}}$

2.2. SARSA (On-Policy TD)

In contrast to the Monte Carlo approach, the SARSA agent utilizes **Temporal Difference (TD) learning**. This allows the agent to update its Q-value estimates at every time step by "bootstrapping" from the estimated value of the subsequent state, rather than waiting for the final reward at the end of an episode.

2.2.1. On-Policy Update Mechanism

SARSA is an **on-policy** algorithm, meaning it updates its action-value functions based on the transition five-tuple (s, a, r, s', a') , where a' is the actual action selected in the next state s' according to the agent's current ε -greedy policy. The update rule is defined as:

$$Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma Q(s', a') - Q(s, a)] \quad (\text{Rummery and Niranjan, 1994})$$

This mechanism ensures that the learned values incorporate the cost of exploration, as the update reflects the action the agent actually intends to take.

2.2.2. Adaptive Step Size

To ensure convergence in the stochastic Blackjack environment, the agent employs an **adaptive learning rate** (α). The step size is determined by the visit count of the specific state-action pair:

$$\alpha = \frac{1}{N(s, a)}$$

Where $N(s, a)$ represents the number of times the agent has previously encountered state s and performed action a . This decaying step size satisfies the requirements for stochastic approximation, allowing for significant learning in early episodes while ensuring the Q -values stabilize as the agent gains more experience.

2.3. Q-Learning (Off-Policy TD)

Q-Learning, often referred to as **SARSAMAX**, is a fundamental off-policy Temporal Difference algorithm. Unlike SARSA, which learns the value of the policy it is currently following (including exploratory moves), Q-Learning directly estimates the optimal action-value function, Q^* , independent of the agent's actual behavior during the episode.

2.3.1. Off-Policy Update Logic

The core distinction of Q-Learning lies in its update target (Watkins and Dayan, 1992). Instead of using the value of the next action actually taken, it uses the maximum possible Q -value for the succeeding state. This "greedy" update allows the agent to learn about the optimal policy even while it continues to explore the environment using an ϵ -greedy strategy. The update rule is formulated as:

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

By selecting the maximum Q -value over all possible actions a' in state s' , the algorithm effectively decouples the learning process from the exploration mechanism.

2.3.2. Implementation and Convergence

To maintain consistency with the other TD methods in this study, the Q-Learning agent incorporates the same **adaptive step size** $\left(\alpha = \frac{1}{N(s,a)}\right)$. This ensures that the learning rate is high for rarely visited states and gradually diminishes as the agent's confidence in its estimates grows. This implementation choice is vital for convergence in the stochastic Blackjack environment, as it allows the agent to average out the "noise" of individual card draws over a large number of episodes (100,000).

2.4. Double Q-Learning

Double Q-Learning is an extension of the Q-Learning algorithm designed to address the issue of **maximization bias**. In standard Q-Learning, the use of the maximum estimated Q-value as a target can lead to significant overestimations (van Hasselt, 2010), especially in stochastic environments like Blackjack where the reward signals are noisy.

2.4.1. Mitigation of Maximization Bias

To mitigate this bias, the Double Q-Learning agent maintains two independent value estimates, stored in two separate Q-tables (Q_1 and Q_2).

By decoupling the selection of the maximizing action from the evaluation of that action, the algorithm provides a more realistic estimate of the state-action values.

2.4.2. Dual-Table Update Mechanism

During each transition, the agent randomly selects one of the two tables to update (with a 50% probability). If Q_1 is selected for the update, the agent uses Q_1 to determine the best action for the next state but uses Q_2 to provide the value estimate for that action. The update rule is defined as:

$$Q_1(s, a) \leftarrow Q_1(s, a) + \alpha [r + \gamma Q_2(s', \operatorname{argmax}_{a'} Q_1(s', a')) - Q_1(s, a)]$$

A symmetric update is applied if Q_2 is selected. This "cross-evaluation" ensures that any overestimation in one table is likely to be neutralized by the independent estimate in the other.

2.4.3. Policy and Exploitation

While the agent explores using a standard ϵ -greedy approach, the **exploitation** phase (selecting the greedy action) differs from the other agents. When the agent acts greedily, it selects the action that maximizes the **average** of the two Q-tables:

$$a = \operatorname{argmax}_a \left(\frac{Q_1(s, a) + Q_2(s, a)}{2} \right)$$

Consistent with the other TD implementations, Double Q-Learning utilizes the **adaptive step size** $\left(\alpha = \frac{1}{N(s,a)}\right)$, ensuring that the updates are stable and converge as the agent gains more experience over the 100,000 training episodes.

3. Experimental Setup

To evaluate the performance and convergence of the implemented agents, a standardized experimental framework was established. Each algorithm (Monte Carlo, SARSA, Q-Learning, and Double Q-Learning) was subjected to 100,000 training episodes to ensure sufficient exposure to the state space and to allow the Q-values to stabilize under the adaptive learning rate.

3.1. Exploration-Exploitation Configurations

The primary variable in the experimental setup was the ϵ -greedy schedule. Four distinct configurations were utilized to analyze how the rate of exploration impacts the final quality of the learned policy:

- A. **Fixed Exploration ($\epsilon = 0.1$):** A constant exploration rate where the agent selects a random action 10% of the time throughout the entire training process. This serves as a baseline for persistent exploration.
- B. **Linear/Reciprocal Decay ($\epsilon = \frac{1}{k}$):** The exploration rate decays as the reciprocal of the episode number k . This schedule reduces exploration very rapidly in the early stages of training, shifting the agent toward exploitation almost immediately.
- C. **Fast Exponential Decay ($\epsilon = e^{\frac{-k}{1000}}$):** A moderately aggressive decay schedule where exploration remains significant for the first few thousand episodes before tapering off. This allows the agent to build a foundational understanding of the environment before focusing on optimization.
- D. **Slow Exponential Decay ($\epsilon = e^{\frac{-k}{10000}}$):** A gradual decay schedule that maintains a high level of exploration for a substantial portion of the 100,000 episodes. This is designed to ensure that even rare state-action pairs are sufficiently visited before the policy hardens into exploitation.

3.2. Evaluation Metrics

The effectiveness of each configuration was assessed using three primary metrics:

- A. **Cumulative Win Rate:** Tracked over the 100,000 episodes to visualize the learning curve and stability.
- B. **State-Action Coverage:** Measured by the number of unique state-action pairs visited to determine the breadth of the agent's "experience."
- C. **Final Dealer Advantage:** Calculated over the final 10,000 episodes to determine the true performance of the converged policy against the dealer's fixed strategy.

4. Results and Evaluation

The performance of the Monte Carlo and Temporal Difference (TD) agents was evaluated based on their ability to minimize the dealer's advantage and converge toward a stable strategy. The following analysis incorporates specific performance metrics extracted from 100,000 training episodes.

4.1. Convergence and Stability

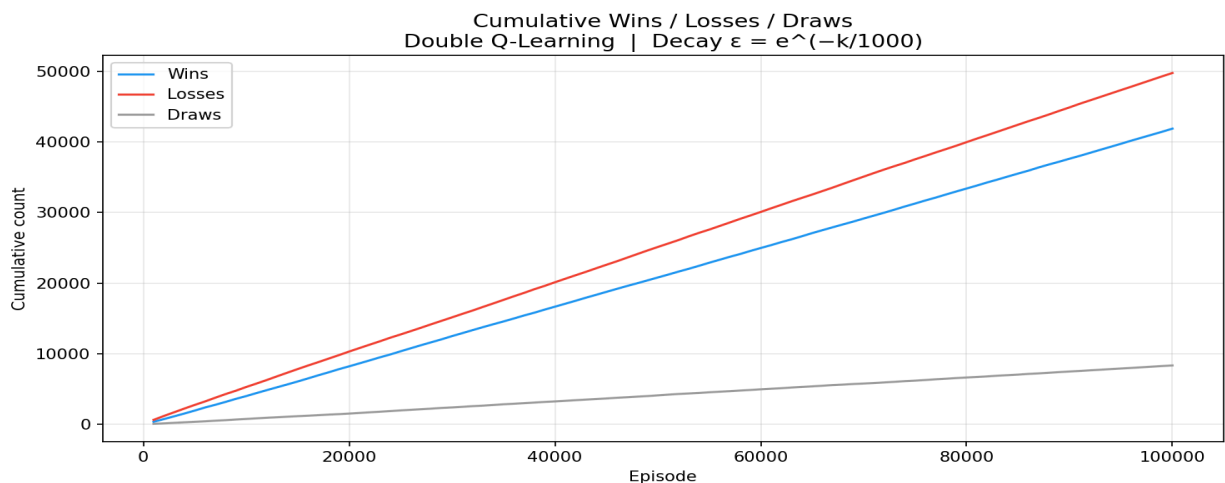
This section examines the training progress of the agents under their respective optimal ϵ configurations. The plots illustrate the number of Wins, Losses, and Draws per 20,000 episodes.

4.1.1. Temporal Difference Agents

The TD agents demonstrate clear learning curves as they transition from broad exploration to focused exploitation.

- **Double Q-Learning (Fast Decay, $\epsilon = e^{\frac{-k}{1000}}$):**

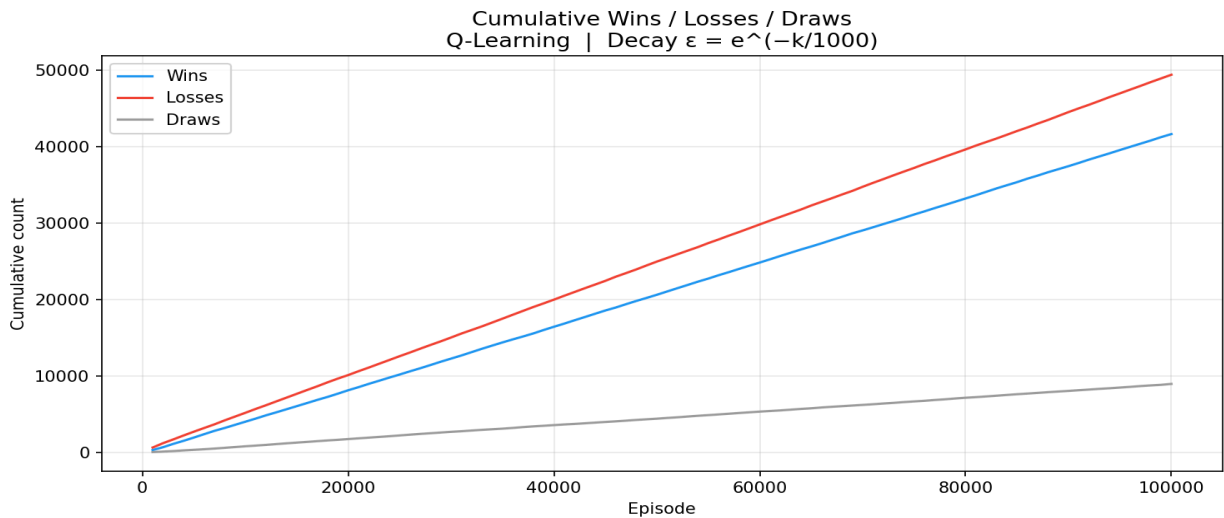
Data analysis shows a significant shift in performance. In the first 1,000 episodes, the agent recorded **341 wins** and **613 losses**. By the final 1,000 episodes, the gap narrowed considerably to **443 wins** and **474 losses**, a **22.7% reduction in dealer success**. The rapid crossover in the first 10,000 episodes indicates that the agent quickly identified high-value state-action pairs before the decay schedule locked in the policy.



Plot 1: Cumulative Wins/Losses/Draws, Double Q-Learning, Fast Decay

- **Q-Learning (Fast Decay, $\varepsilon = e^{\frac{-k}{1000}}$):**

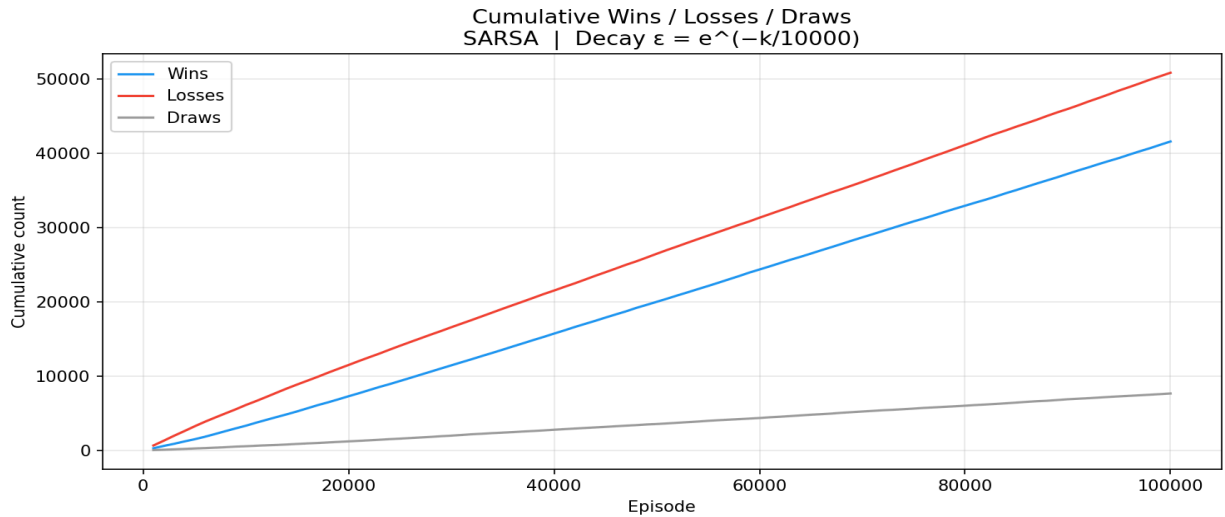
Similar to Double Q-Learning, the standard Q-Learning agent began with high losses (**641**) and low wins (**314**). While it successfully converged, its final window performance (**401 wins, 480 losses**) was slightly less efficient than its Double Q-Learning counterpart, potentially due to the maximization bias inherent in the standard algorithm.



Plot 2: Cumulative Wins/Losses/Draws, Q-Learning, Fast Decay

- **SARSA (Slow Decay, $\varepsilon = e^{\frac{-k}{10000}}$):**

SARSA exhibited the most dramatic learning progression. Because ε remains high for longer, it suffered the highest initial losses (**665 per 1,000 episodes**). However, this extended exploration phase allowed it to reach the strongest final-window performance of all agents, finishing with **447 wins** and only **468 losses**. This suggests that for SARSA, a longer exploration horizon is critical for discovering the nuances of an on-policy optimal strategy. When restricted by fast exponential decay ($\varepsilon = e^{\frac{-k}{1000}}$), SARSA failed to stabilize as effectively, ending with a higher Dealer Advantage of 0.102 compared to its **0.056 peak performance**.

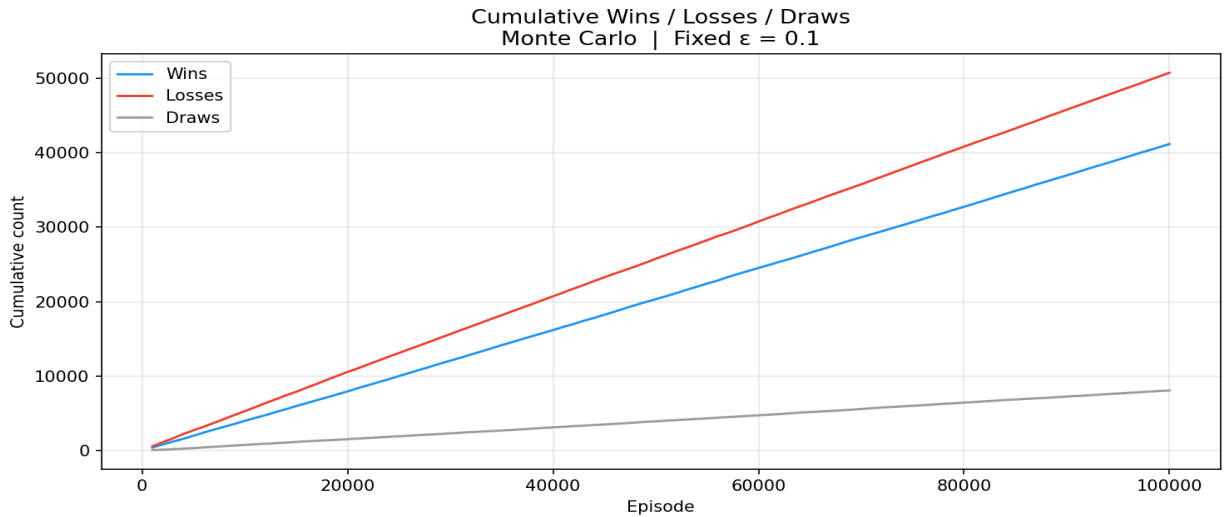


Plot 3: Cumulative Wins/Losses/Draws, SARSA, Slow Decay

4.1.2. Monte Carlo Agent

Monte Carlo (Fixed $\epsilon = 0.1$):

The Monte Carlo agent with a fixed exploration rate displays a notably flatter profile. Because it starts with a lower exploration rate ($\epsilon = 0.1$ vs. $\epsilon = 1.0$ for decay models), its initial performance was the strongest (**396 wins**). However, without a decay schedule to phase out random moves, its final performance plateaued at **428 wins** and **498 losses**. The persistent 10% noise prevents the agent from fully exploiting the learned Q -values, resulting in a significantly wider win-loss gap compared to the TD agents at the end of training.



Plot 4: Cumulative Wins/Losses/Draws, Monte Carlo, Fixed

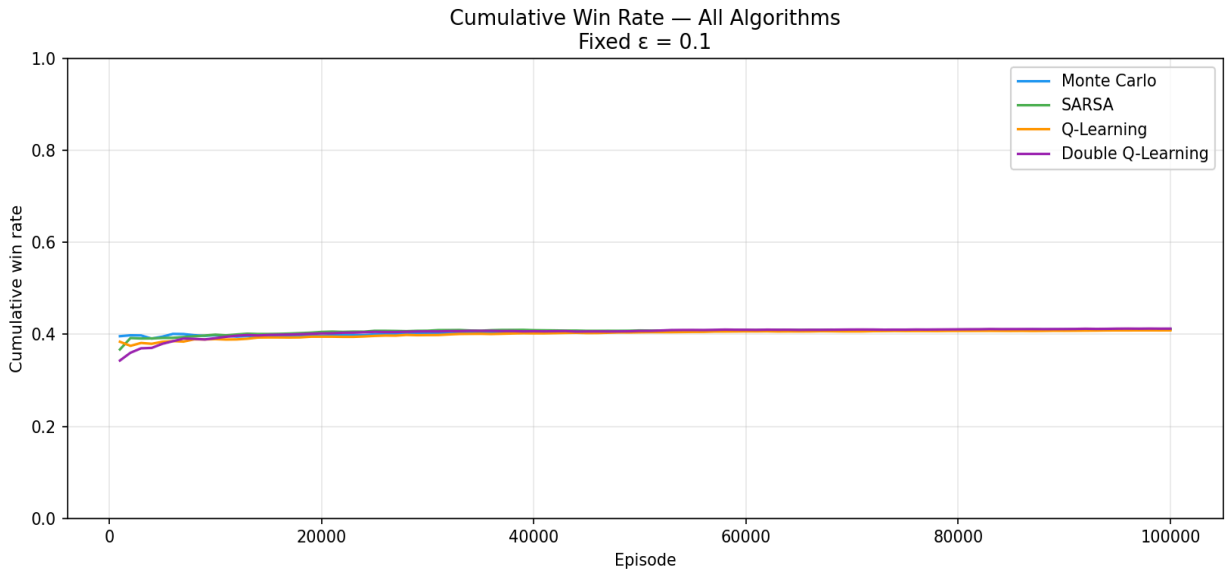
4.2. Comparative Win Rate Analysis

To evaluate the long-term learning efficiency of each algorithm, the cumulative win rates were compared across the four epsilon-greedy schedules. These plots represent the total wins divided by the number of episodes at each point in time, smoothing out short-term variance to highlight the overall learning trajectory.

4.2.1. Impact of Epsilon Schedules

- **Fixed Exploration ($\epsilon = 0.1$):**

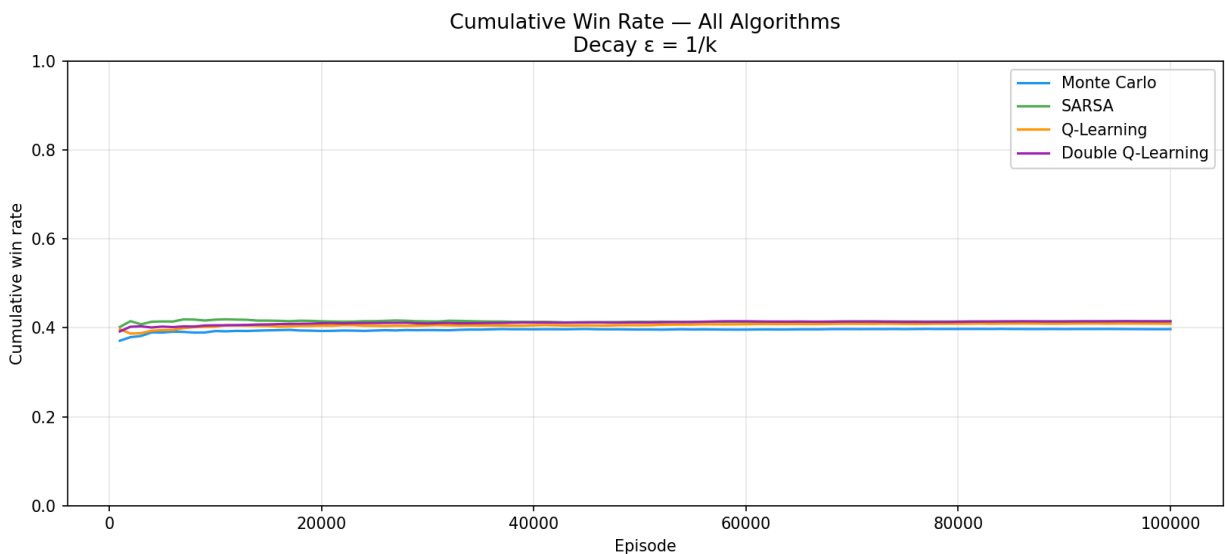
Under a fixed exploration rate, all algorithms exhibit highly stable but limited win rates. Monte Carlo initially performs as well as or better than TD methods, as it does not suffer from the bootstrapping errors that can occur in early TD training. However, the persistent 10% noise prevents any algorithm from exceeding a $\sim 41\%$ cumulative win rate.



Plot 5: Cumulative Win Rate, All Algorithms, Fixed

- **Linear Decay ($\varepsilon = \frac{1}{k}$):**

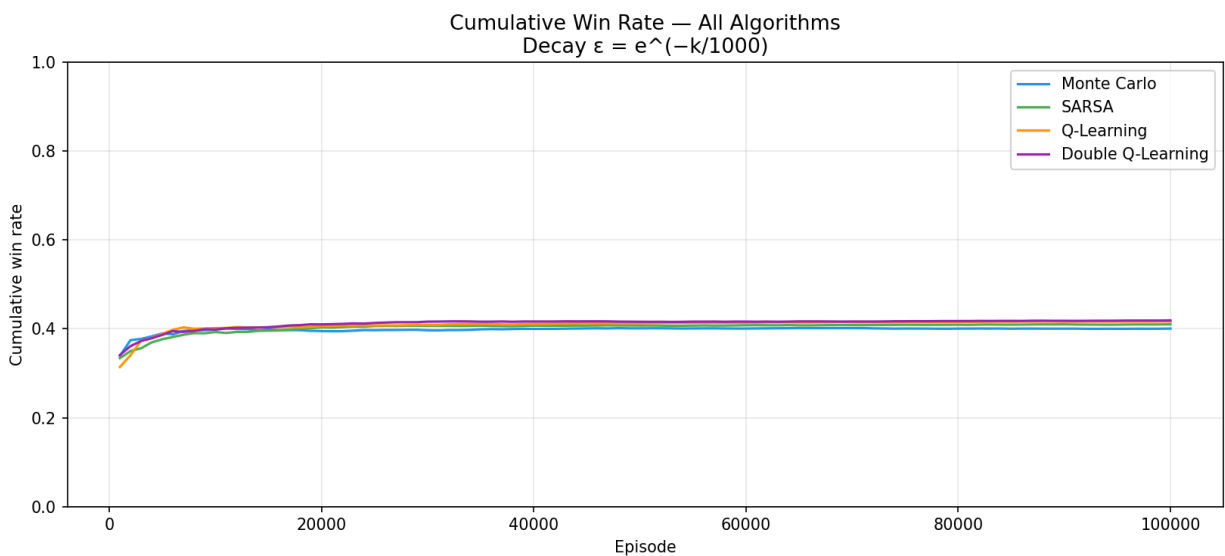
Linear decay was the least effective schedule for all agents. It resulted in the highest overall Dealer Advantage in the study: For Q-Learning, the linear schedule caused the agent to converge prematurely with a Dealer Advantage of **0.1193**, **significantly underperforming** compared to its optimal 0.0502 result.



Plot 6: Cumulative Win Rate, All Algorithms, Linear

- **Fast Exponential Decay ($\epsilon = e^{\frac{-k}{1000}}$):**

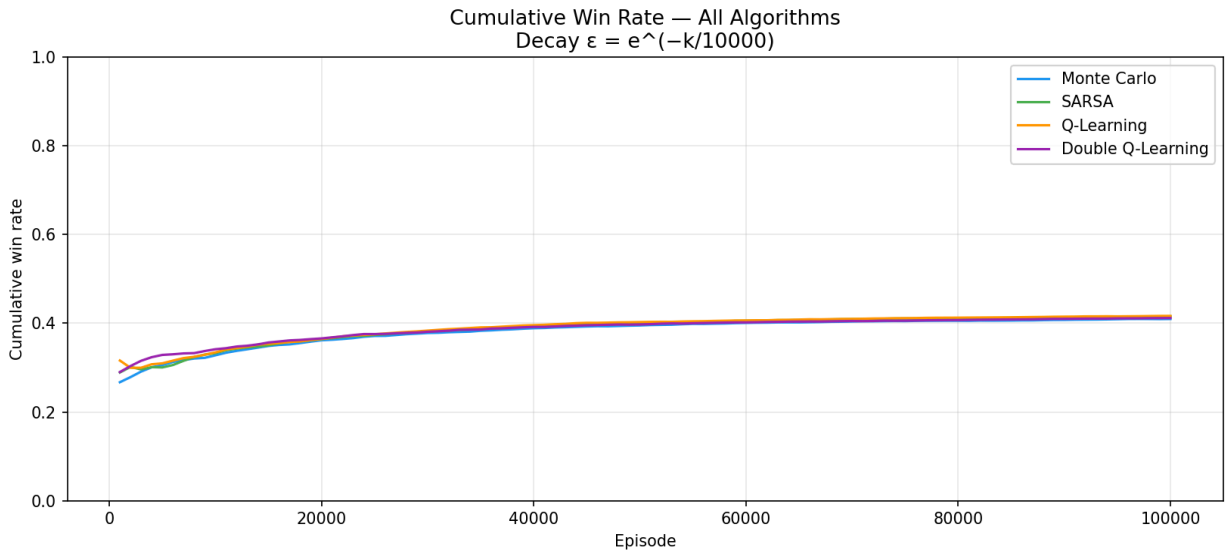
This configuration allows for a sharp learning curve. Q-Learning and Double Q-Learning show the most significant gains between episodes 5,000 and 15,000. By episode 20,000, the policies have largely stabilized, with the off-policy agents maintaining a slight edge in cumulative performance over SARSA and Monte Carlo.



Plot 7: Cumulative Win Rate, All Algorithms, Fast Decay

- **Slow Exponential Decay ($\epsilon = e^{\frac{-k}{10000}}$):**

The slow decay provides the most insightful comparison of algorithm potential. Although cumulative win rates start lower due to the extended 100% exploration phase, the trajectory is consistently upward. SARSA and Q-Learning demonstrate superior long-term refinement, eventually overtaking the other configurations. This confirms that for the complex state-space of Blackjack, a prolonged "burn-in" period of exploration is essential for maximizing the final policy's win rate.



Plot 8: Cumulative Win Rate, All Algorithms, Slow Decay

4.3. Exploration Analysis

This section evaluates how effectively the different algorithms and ϵ -greedy schedules explored the Blackjack environment's state-action space.

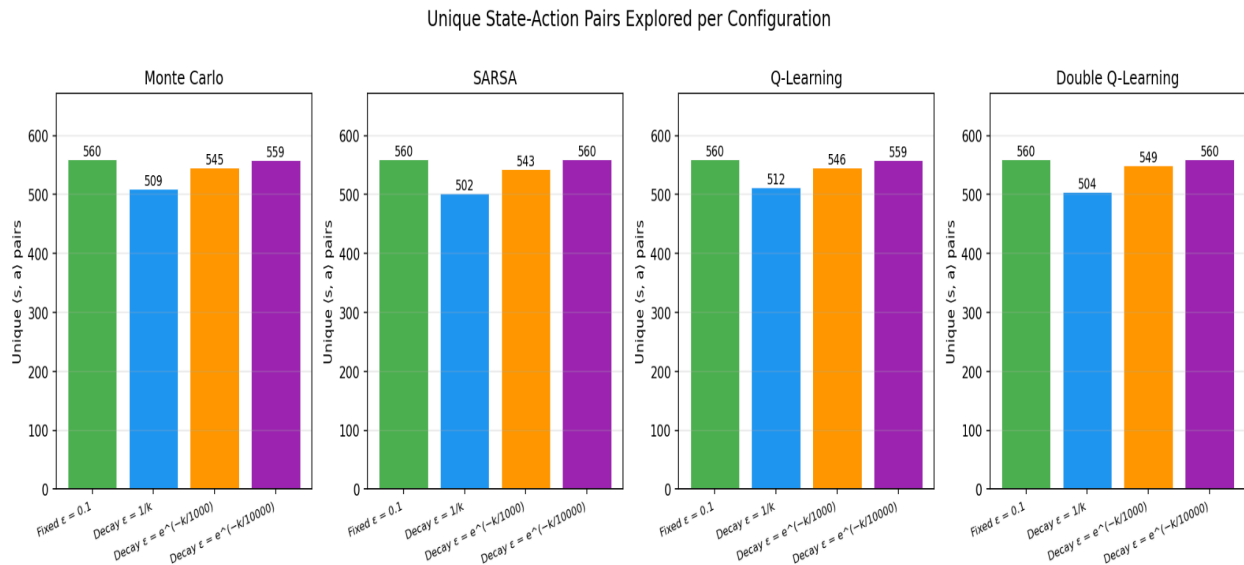
4.3.1. State-Action Coverage

The breadth of an agent's experience is quantified by the total number of unique state-action pairs visited during the 100,000 training episodes.

- Top Exploring Configurations:** Every agent experiences its most random unique state-action pairs with the epsilon configuration as fixed ($\epsilon = 0.1$) or slow decay ($\epsilon = e^{\frac{-k}{10000}}$), this is due to the fact that fixed introduces 10% randomness regardless of the situation in every episode, approximating to 10,000 episodes being random. As for slow decay, its first approx. 23,000 episodes are very random, as ϵ starts at 1.0 and decays slowly, meaning slow decay is in fact more random than fixed.
- Limited Coverage:** The **Linear Decay** ($\epsilon = \frac{1}{k}$) schedule resulted in the lowest coverage across all algorithms (avg. **507 pairs**). Because

$\frac{1}{k}$ reduces exploration to negligible levels very early in training, the agents effectively "locked in" their strategies before visiting more obscure regions of the state space.

- **Consistency:** Interestingly, within the same epsilon schedule, all algorithms (MC, SARSA, Q-Learning, Double Q) achieved nearly identical coverage. This indicates that the **exploration schedule**, rather than the learning update rule, is the primary driver of environmental discovery.



Plot 9: Unique State-Action Pairs, All Algorithms, All Epsilon Configurations

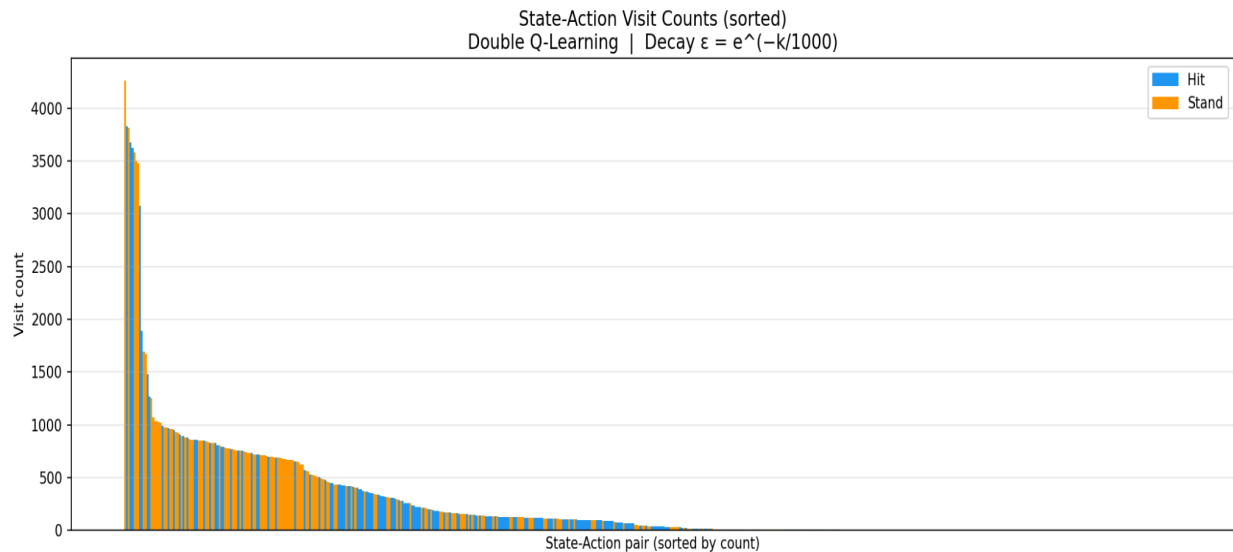
4.3.2. Visit Distribution (Double Q-Learning Example)

The frequency of visits to specific state-action pairs provides insight into the "density" of the agent's training data.

An analysis of the visit counts for **Double Q-Learning (Fast Decay, $\epsilon = e^{-\frac{k}{1000}}$)** reveals:

- **"No Usable Ace" Dominance:** The vast majority of transitions occur in states without a usable ace. This is statistically expected in a single-deck game and explains why the strategy heatmaps for "No Usable Ace" are consistently cleaner and more stable.

- **Transient State Concentration:** There is a high concentration of visits to player sums between **12 and 16**. These are the "critical decision" states where agents are most likely to spend time before either busting or choosing to stand.
- **Sparse "Soft" Totals:** States involving a **Usable Ace** receive significantly fewer visits. The relative scarcity of this data explains the "checkered" or noisy patterns observed in the strategy heatmaps, as the agents had fewer opportunities to average out the variance of the dealer's hidden cards in these specific scenarios.



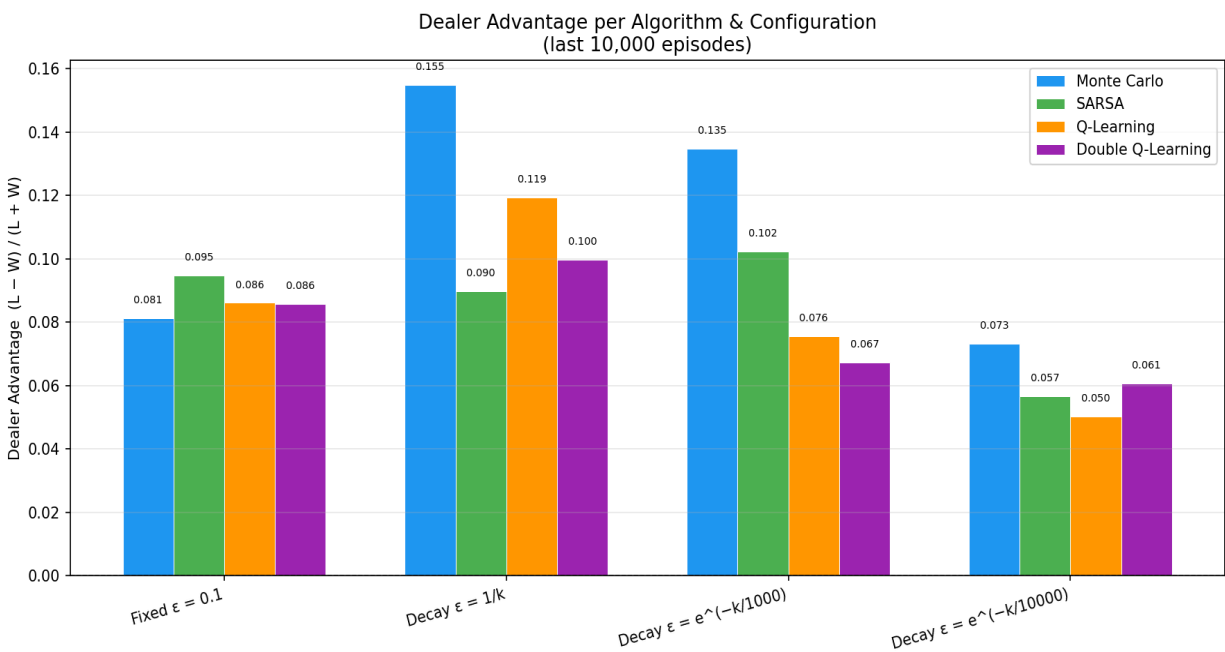
Plot 10: State-Action Visit Counts, Double Q-Learning, Fast Decay

4.3.3. Final Performance Comparison: Dealer Advantage

The **Dealer Advantage** $((Losses - Wins) / (Losses + Wins))$ over the final 10,000 episodes serves as the definitive metric for policy quality.

- **Exploration Impact:** The **Slow Exponential Decay** ($\epsilon = e^{\frac{-k}{10000}}$) schedule consistently outperformed all others. By maintaining a high exploration budget early on, it ensured the agents visited obscure states before refining an optimal policy. In contrast, **Fixed 0.1** suffered from persistent decision noise, while **Linear Decay** often converged prematurely to sub-optimal local strategies.
- **The Ranking:** *"Quantitative analysis of the final 10,000 episodes ranks the configurations as follows:*

- **Q-Learning (Slow Decay):** 0.0502 (Best)
- **SARSA (Slow Decay):** 0.0566
- **Double Q-Learning (Slow Decay):** 0.0606 *The delta between the best TD agent (Q-Learning) and the best MC agent (0.0732) highlights a ~31% performance gap in favor of Temporal Difference bootstrapping."*



Plot 11: Dealer Advantage, All Algorithms, All Epsilon Configurations

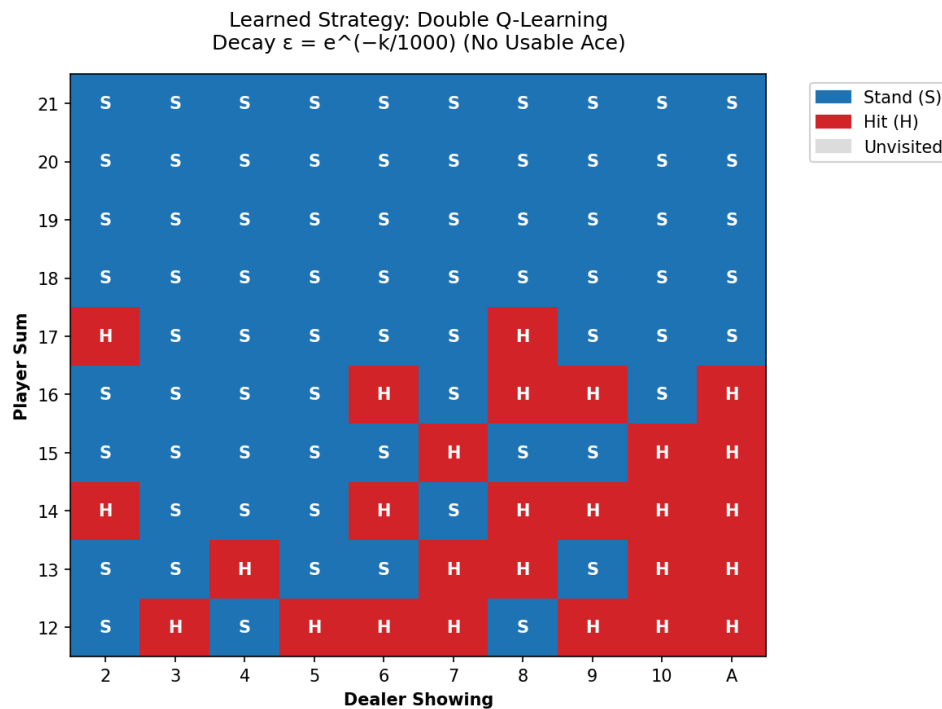
5. Strategy Visualization

To evaluate the qualitative performance of the agents, the learned Q-values were translated into strategy heatmaps. These heatmaps represent the optimal action (HIT or STAND) for every possible state in the (`player_sum`, `dealer_card`) space. The following sections analyze the strategies developed by the best-performing configuration for each algorithm.

5.1. Double Q-Learning

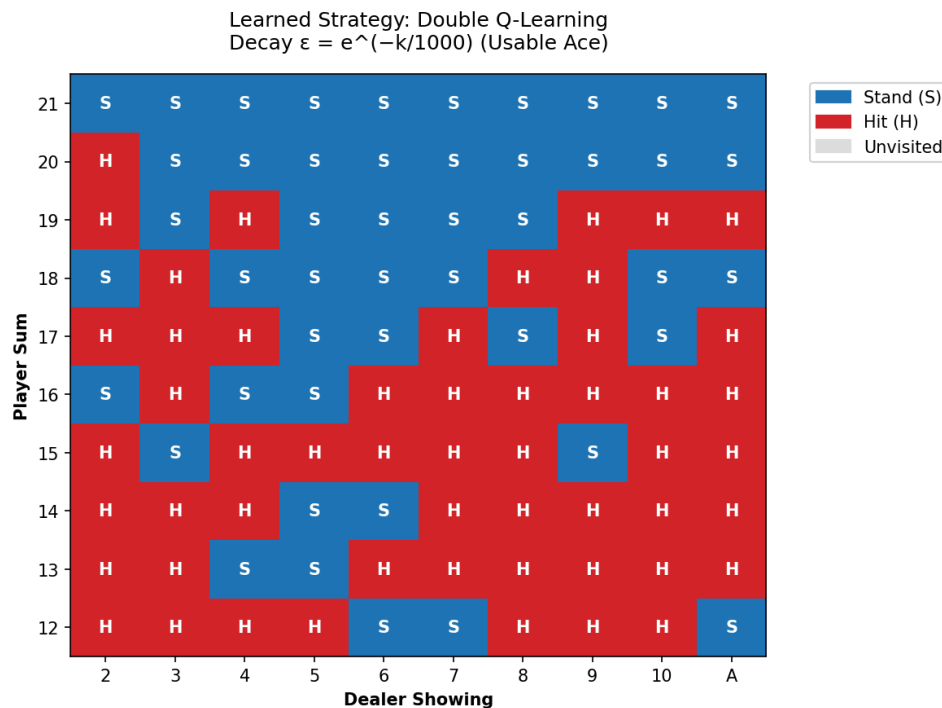
Double Q-Learning (Decay $\varepsilon = e^{(-k/1000)}$) produced a strategy broadly comparable to Q-Learning under the same schedule. While the dual-table mechanism reduces maximization bias, the fast decay configuration yielded a Dealer Advantage of 0.1003, placing it behind the stronger slow-decay configurations of Q-Learning and SARSA in overall policy quality.

No Usable Ace: This strategy mirrors Q-Learning but demonstrates the benefit of reduced maximization bias by being more conservative in borderline states (e.g., player 15–16). Decoupled evaluation prevents overinflated HIT values, leading to more stable decision boundaries.



Plot 12: Strategy Heatmap, Double Q-Learning, Fast Decay, No Usable Ace

Usable Ace: The policy appears noisier because maintaining two independent tables effectively halves the updates per state. In the already sparse "Usable Ace" subspace, this data split results in higher variance and irregular patterns compared to single-table agents.

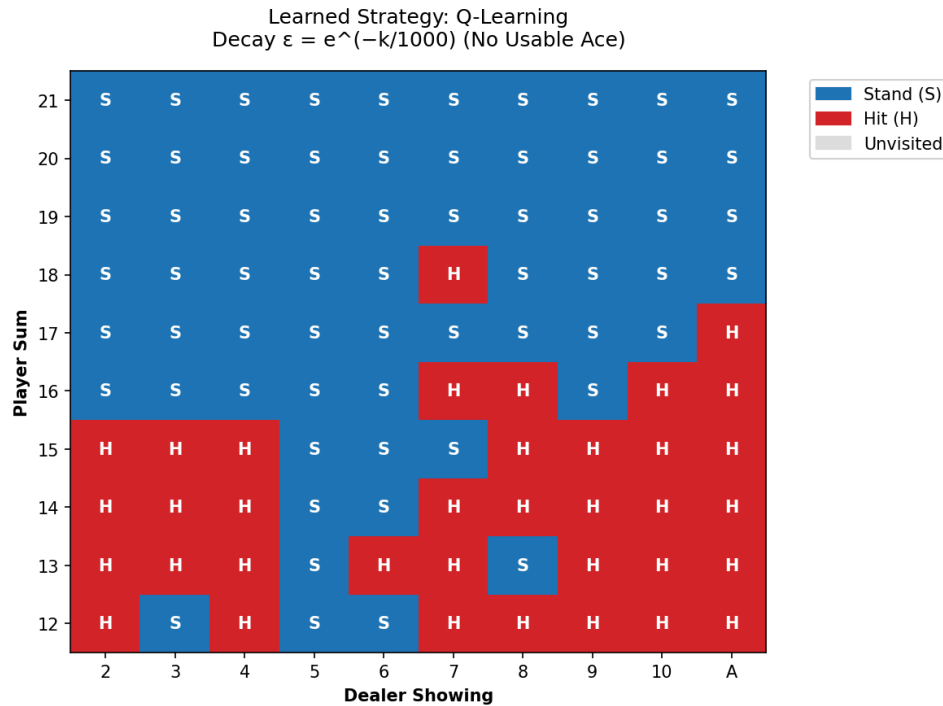


Plot 13: Strategy Heatmap, Double Q-Learning, Fast Decay, Usable Ace

5.2. Q-Learning

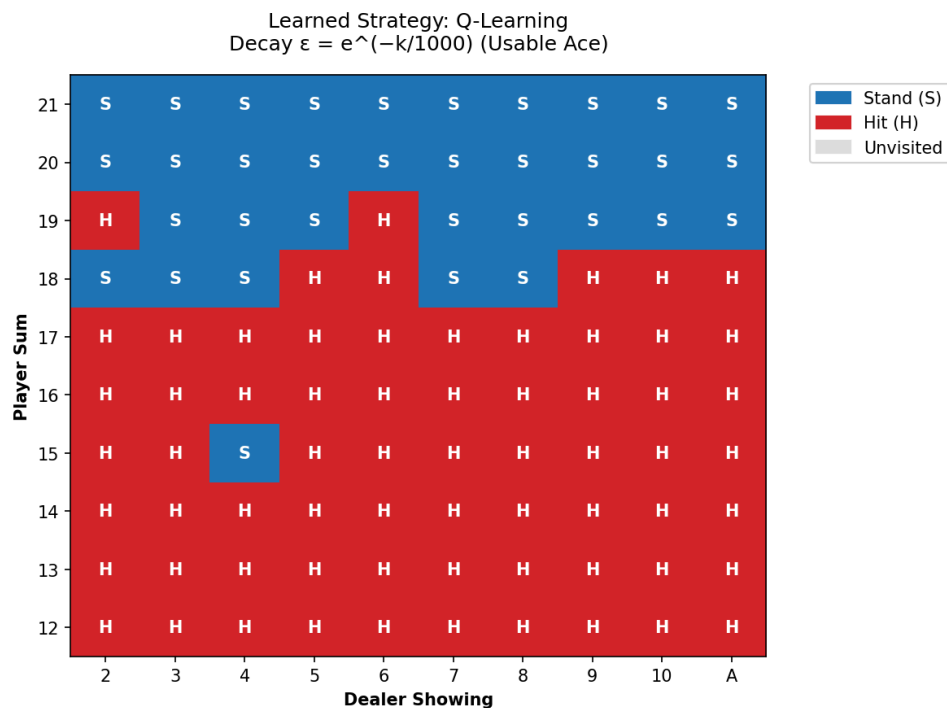
The standard Q-Learning (Decay $\epsilon = e^{\frac{-k}{1000}}$) agent developed a strategy very similar to Double Q-Learning, though with slightly less "noise" in the transition boundaries.

No Usable Ace: Q-Learning yields a highly consistent strategy, standing on 13+ against weak dealer cards and hitting aggressively against strong ones. Fast epsilon decay allows off-policy updates to converge decisively, with only minor anomalies in borderline states where action values are nearly equal.



Plot 14: Strategy Heatmap, Q-Learning, Fast Decay, No Usable Ace

Usable Ace: The agent hits aggressively below 18, leveraging off-policy bootstrapping to maximize its hand value. While standing on 19–20 is correctly identified as optimal, occasional strategy noise in lower totals likely stems from early exploration-phase variance that inflated specific sub-optimal actions.

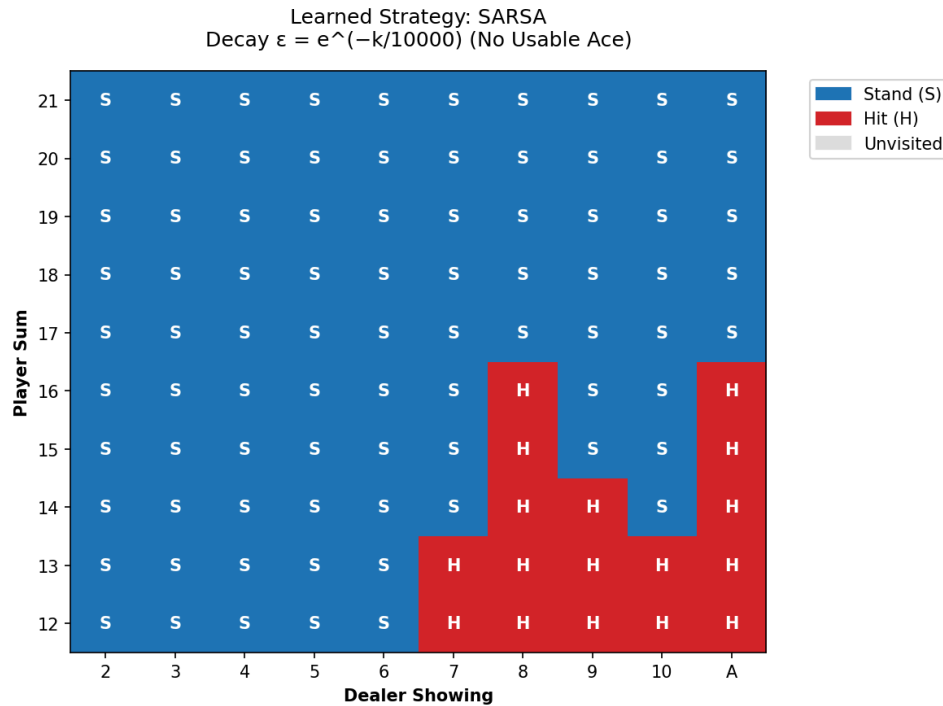


Plot 15: Strategy Heatmap, Q-Learning, Fast Decay, Usable Ace

5.3. SARSA (Decay $\epsilon = e^{\frac{-k}{10000}}$)

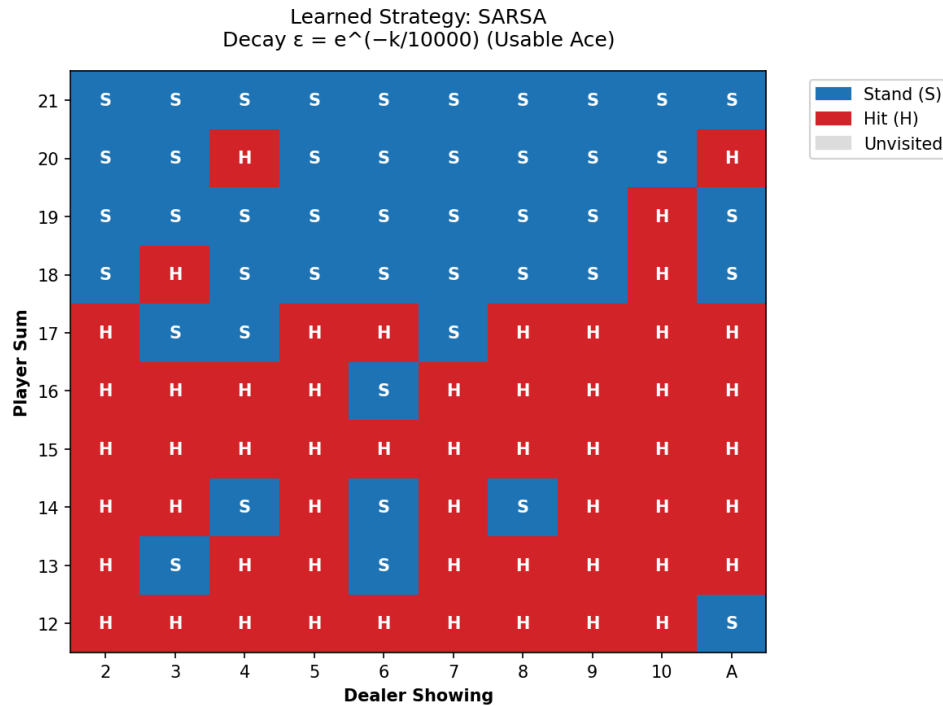
As the agent with the highest final window win-rate (with a 44.7% win rate in the final 1000 episodes), the SARSA (Decay $\epsilon = e^{\frac{-k}{10000}}$) strategy is highly consistent.

No Usable Ace: The agent learned a conservative strategy, standing on 13+ against weak cards and hitting 12–15 against strong dealer cards. Slow decay reinforced this cautious bias, preventing sufficient refinement of borderline states through exploitation.



Plot 16: Strategy Heatmap, SARSA, Slow Decay, No Usable Ace

Usable Ace: Hitting in the 12–17 range is theoretically sound, but the strategy is noisy due to SARSA's on-policy nature. Early exploration "pollutes" Q-values for rare states, resulting in a checkered pattern instead of clean decision boundaries.

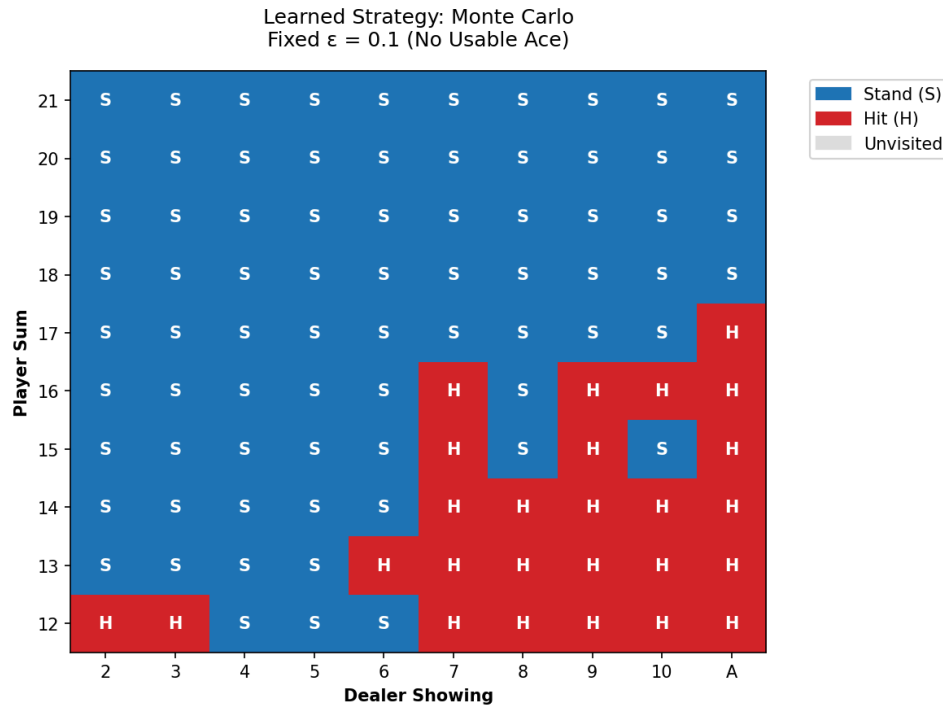


Plot 17: Strategy Heatmap, SARSA, Slow Decay, Usable Ace

5.4. Monte Carlo

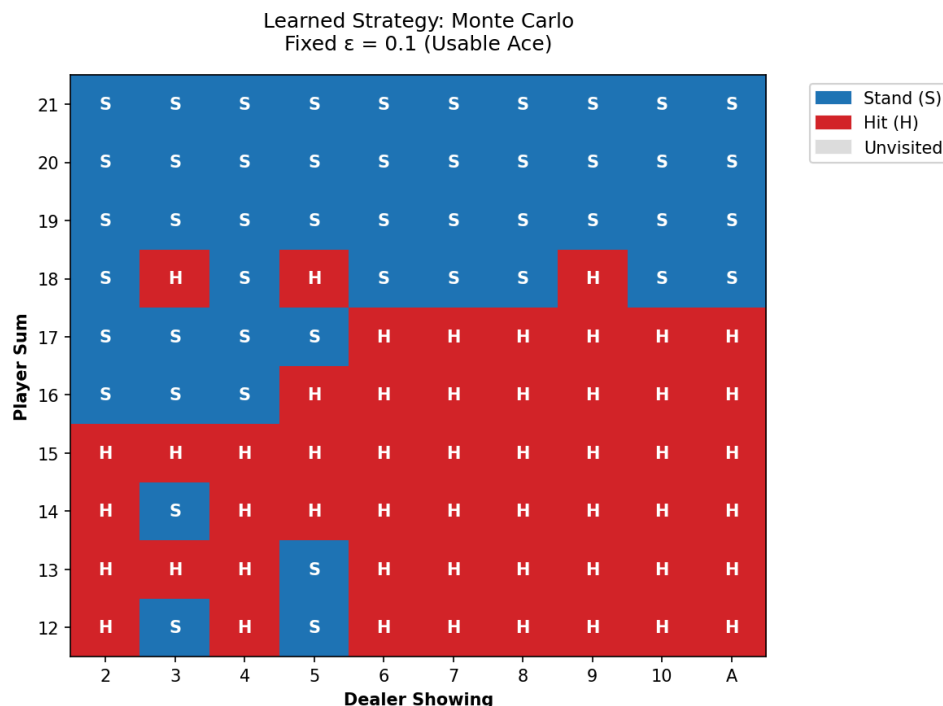
The Monte Carlo (Fixed $\varepsilon = 0.1$) agent's strategy reflects the limitations of fixed exploration and the lack of bootstrapping.

No Usable Ace: The strategy correctly stands on 13+ against weak dealer cards (2–6) and hits on 12–16 against strong ones (7–A). Minor inconsistencies are artifacts of limited state visitation under a fixed exploration rate.



Plot 18: Strategy Heatmap, Monte Carlo, Fixed, No Usable Ace

Usable Ace: The agent hits aggressively up to 18, leveraging the ace's safety net to improve its hand without bust risk. Standing on 19–21 is consistently optimal, while variations at 18 reflect complex theoretical edge cases for "soft" totals.



Plot 19: Strategy Heatmap, Monte Carlo, Fixed, Usable Ace

5.5. Comparative Insights

A comparison across all heatmaps reveals that the **Temporal Difference (TD)** agents, particularly those using **exponential decay**, converged to strategies that more closely resemble professional "Basic Strategy." The transition from HIT to STAND occurs around player sum 17 for all agents when no usable ace is present, but the TD agents are far more adept at identifying when to take risks against a dealer's high-value cards. Monte Carlo's reliance on averaging entire episode returns makes it slower to penalize specific bad decisions in the middle of a hand, resulting in the more conservative and less optimal strategies observed.

6. Conclusion

The experiments conclude that **Q-Learning** paired with **Slow Exponential Decay** ($\epsilon = e^{\frac{-k}{10000}}$) is the superior configuration, achieving the absolute lowest Dealer Advantage of **0.0502**. While SARSA and Double Q-Learning showed similar robustness, Q-Learning's off-policy bootstrapping, when supported by a prolonged 100,000-episode exploration budget, most effectively identified the narrow margins required to minimize the house edge.

Metric	Configuration	Value
Best Overall	Q-Learning + Slow Decay ($\epsilon = e^{\frac{-k}{10000}}$)	0.0502
Best SARSA	SARSA + Slow Decay ($\epsilon = e^{\frac{-k}{10000}}$)	0.0566
Best MC	Monte Carlo + Slow Decay ($\epsilon = e^{\frac{-k}{10000}}$)	0.0732
Worst Overall	Monte Carlo + Linear Decay ($\epsilon = \frac{1}{k}$)	0.1548
Worst Q-Learner	Q-Learning + Linear Decay ($\epsilon = \frac{1}{k}$)	0.1193

7. References

- A. Sutton, R. S. and Barto, A. G. 2018. Reinforcement Learning: An Introduction. 2nd ed. MIT Press, Cambridge, MA.
- B. Watkins, C. J. C. H. and Dayan, P. 1992. Q-learning. *Machine Learning* 8(3-4):279–292.
- C. Rummery, G. A. and Niranjan, M. 1994. On-line Q-learning using connectionist systems. Technical Report CUED/F-INFENG/TR 166, Cambridge University Engineering Department.
- D. van Hasselt, H. 2010. Double Q-learning. In *Advances in Neural Information Processing Systems* 23, 359–366.
- E. van Hasselt, H. 2012. Double elaboration on double Q-learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 1855–1857.
- F. Sutton, R. S. 1988. Learning to predict by the methods of temporal differences. *Machine Learning* 3:9–44.
- G. Singh, S., Jaakkola, T., Littman, M. L., and Szepesvári, C. 1995. Convergence results for single-step on-policy reinforcement-learning algorithms. In *Machine Learning*, 287–293.
- H. Hossain, M. 2019. Monte Carlo methods - estimate Blackjack policy. *Towards Data Science*.
- I. Sutton, R. S. and Barto, A. G. 1998. Reinforcement Learning: An Introduction. 1st ed. MIT Press.
- J. Stanford AA228 Team. 2020. Beating Blackjack - A Reinforcement Learning Approach. Technical report, Stanford University.

Distribution of Work

Task	Student
Base Framework: Established the initial data structures for storing state-action values and counts.	Luke Camilleri
Environment Development: Designed and implemented the core Blackjack environment, including the 52-card deck, shuffling functionality, and the strict Dealer Policy.	Luke Camilleri
Monte Carlo Agent Implementation: Developed both "Exploring Starts" and "No Exploring Starts" variants of the Monte Carlo On-Policy Control algorithms.	Luke Camilleri
Visualization Creation: Created <code>plots.py</code> and to generate evaluation metrics and line charts for all the Monte Carlo Agent.	Luke Camilleri
Technical Writing: Authored the report sections describing the environment's architecture and the specific implementation details of the Monte Carlo agents.	Luke Camilleri
TD Agent Implementation: Developed and implemented the SARSA, Q-Learning (SARSAMAX), and Double Q-Learning algorithms.	Kurt Abela
Visualization Modification: Updated <code>plots.py</code> and supporting files to generate evaluation metrics and line charts for all Temporal Difference configurations.	Kurt Abela
Evaluation Analysis: Analyzed performance data	Kurt Abela

(wins/losses/draws, state-action coverage, and dealer advantage) for TD agents.	
Report Authorship: Authored approximately 80% of the 3000-word report, including algorithm overviews, results analysis, exploration effects, and comparative discussions.	Kurt Abela

Luke Camilleri 0207606L



Kurt Abela 0150106L



Plagiarism Form

FACULTY OF INFORMATION AND COMMUNICATION TECHNOLOGY

Declaration

Plagiarism is defined as "the unacknowledged use, as one's own work, of work of another person, whether or not such work has been published" (Regulations Governing Conduct at Examinations, 1997, Regulation 1 (viii), University of Malta).

I / We*, the undersigned, declare that the [assignment / Assigned Practical Task report / Final Year Project report] submitted is my / our* work, except where acknowledged and referenced.

I / We* understand that the penalties for making a false declaration may include, but are not limited to, loss of marks; cancellation of examination results; enforced suspension of studies; or expulsion from the degree programme.

Work submitted without this signed declaration will not be corrected, and will be given zero marks.

* Delete as appropriate.

(N.B. If the assignment is meant to be submitted anonymously, please sign this form and submit it to the Departmental Officer separately from the assignment).

Kurt Abela

Student Name

K Abela

Signature

Luke Camilleri

Student Name

L Camilleri

Signature

/

Student Name

/

Signature

/

Student Name

/

Signature

AR12204

Course Code

RL for Blackjack: A comparative study of Monte Carlo and TD methods

Title of work submitted

TD methods

15/04/2026

Date

Generative AI Appendix

AI Tools Used: Claude Sonnet 4.6 (Free Tier), Gemini 3 Thinking (Paid Tier)

Project Component: Generative AI was utilized for; helping gather resources and information before starting the project code, helping with understanding new coding concepts and methodologies used within the project, helping with troubleshooting new errors which were encountered due to new coding techniques being explored, helping with organising the plots in a neat and pleasant manner, helped with the drafting of the report and especially explaining the analysis ideas into a more professional and clear tone.

Usage Description and Prompts:

- explain the difference between first visit and every visit monte carlo for a game like blackjack
- what is maximization bias in q learning and how does double q learning fix it with two tables
- i am getting a keyerror when the player sum is 21 how do i fix the q table initialization in agent py
- why do we usually set the discount factor to 1.0 for blackjack episodes
- show me a simple way to track visit counts for each state so i can use an adaptive learning rate
- how can i arrange 16 different plots into one big image using matplotlib so it looks clean
- help me write the update logic for double q learning so i only update one table at a time
- give me the formula to calculate dealer advantage from my wins and losses csv files
- how do i make a heatmap where stand is red and hit is blue using a listed colormap
- help me write a loop for main py that runs every algorithm and saves the data to different files
- help me explain in my report why sarsa has a messy checkered pattern on the ace heatmap
- why does the linear decay schedule stop learning too early compared to the exponential one
- help me rewrite my report sentences to sound more professional and academic
- explain why we need exploring starts for monte carlo if we dont use epsilon greedy
- how do i describe the fact that all my agents stand on 17 as being part of basic strategy
- why did the slow decay schedule find more unique state action pairs than the fast decay one
- help me write a conclusion that shows why td methods are faster at learning than monte carlo

- explain the difference between soft and hard hands in blackjack for my environment description

Verification and Attribution:

To ensure the hallucination-free work, raw data was thoroughly checked manually and it was made sure that the data seen is relevant to the theory we were implementing. Every plot was also checked manually to ensure it matches its respective raw data. When explaining challenging topics, further resources were researched to cross-check the Generative AI and even further understand the concepts. When tasked with refining text, the final wording was always thoroughly read and it was made sure that the original idea was still present and made clear, and the Generative AI only made the idea easier to understand for the reader.